

Trilha		Fórum
✓ ebook	01:45	
Desnormalização e Normalização de Dados	02:09	
✓ ebook	02:02	
View Temporárias vs. Tabelas Persistentes	02:10	
✓ ebook	02:09	
Executando SQL com PySpark	02:10	
✓ ebook	02:09	
Uso de Built-in Functions no PySpark	02:17	
pdf	06:10	
Projeto 3 - Pipeline de Limpeza e Transformação Para Aplicações de IA com PySpark SQL	02:50	
Projeto 3 - Visão Geral	02:17	
Projeto 3 - Experimentação x Execução	06:10	
Projeto 3 - Estrutura do Projeto	02:50	
Projeto 3 - Como Realizar Experimentação com o Cluster		



Executando SQL com PySpark

Executar **SQL** com **PySpark** é uma maneira poderosa de interagir com conjuntos de dados distribuídos, aproveitando a familiaridade da linguagem SQL enquanto utiliza a infraestrutura de processamento paralelo do Apache Spark.

Isso permite executar consultas complexas sobre grandes volumes de dados de maneira eficiente.

✓ Configurando o Ambiente PySpark

Para começar a executar SQL com PySpark, você primeiro precisa configurar o ambiente Spark:

1. Inicialização da Sessão Spark: Crie uma sessão Spark, que é o ponto de entrada para todas as funcionalidades do Spark.

```
>> from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Exemplo de Aplicação SQL") \
    .getOrCreate()
```

✓ Trabalhando com DataFrames

2. Criação de DataFrame: Você pode criar DataFrames lendo dados de várias fontes como CSV, Parquet, sistemas de banco de dados ou diretamente de RDDs.

```
>> df = spark.read.csv("caminho_para_arquivo.csv", header=True,
inferSchema=True)
```

✓ Utilizando SQL

3. Registrando DataFrame como View Temporária: Antes de executar consultas SQL, você deve registrar o DataFrame como uma view temporária.

```
>> df.createOrReplaceTempView("nome_view")
```

4. Executando SQL:

Agora, você pode usar o método **sql** da sessão Spark para executar consultas SQL diretamente na view registrada.

```
>> result_df = spark.sql("SELECT * FROM nome_view WHERE coluna1 >
100")
```

Você pode realizar todas as operações **SQL** padrão, como **SELECT**, **JOIN**, **GROUP BY** e outras operações de agregação.

✓ Exemplos de Consultas SQL

Seleção e Filtros

```
>> spark.sql("SELECT coluna1, coluna2 FROM nome_view WHERE
coluna3 BETWEEN 10 AND 20")
```

Agregações

```
>> spark.sql("SELECT COUNT(*), AVG(coluna1) FROM nome_view
GROUP BY coluna2")
```

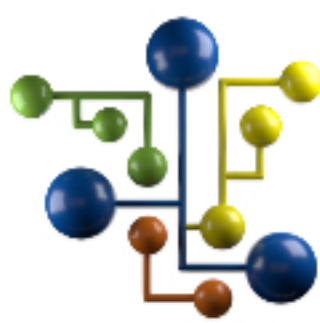
Joins

```
>> df2.createOrReplaceTempView("outra_view")
spark.sql("SELECT a.*, b.* FROM nome_view a JOIN outra_view b ON
a.id = b.id")
```

✓ Dicas para SQL no PySpark

- **Uso de Views Temporárias vs. Tabelas Persistentes:** Para análises ad-hoc, use views temporárias. Para resultados que você precisa reutilizar ou compartilhar entre várias sessões ou usuários, considere gravar o resultado em tabelas persistentes.
- **Desempenho:** Lembre-se de que o Spark executa otimizações internamente, então algumas vezes as operações SQL podem ser mais eficientes do que as manipulações equivalentes de DataFrame usando APIs do PySpark.
- **Integração com outras fontes de dados:** O PySpark permite ler e escrever dados de e para muitas fontes, facilitando a integração com sistemas existentes.

✓ Usar SQL com PySpark não só aumenta a acessibilidade dos dados para usuários que já estão familiarizados com SQL, mas também tira proveito da escalabilidade e do desempenho do Spark para processamento de grandes volumes de dados.



Equipe DSA

Continue trilhando uma excelente jornada de aprendizagem.

?

Suporte

?

Suporte

?

Suporte

?

Suporte

?

Suporte

?

Suporte

?

Suporte

?

Suporte

?

Suporte

?

Suporte